

CSE 579 Homework 2

Hisham Bhatti

1 Code Overview

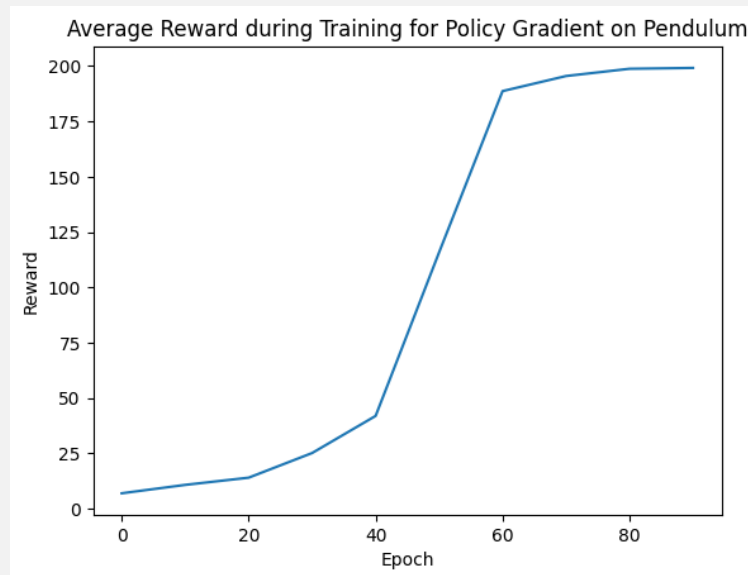
2 Policy Gradient

1. Fill in the blanks in the code marked with TODO in the `policy_gradient` function in `policy_gradient.py`

See Gradescope submission for code.

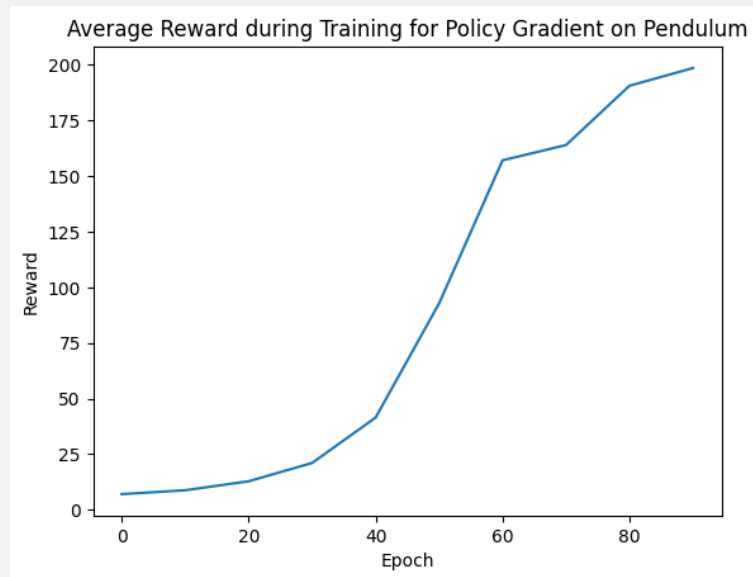
2. Plot the reward during the training with default hyperparameters. Report the final success rate and average reward using the `evaluate` function that is provided to you.

Environment: Pendulum



(Final) Success Rate: 0.63
Average Reward (success only): 200.0
Average Reward (all): 191.34

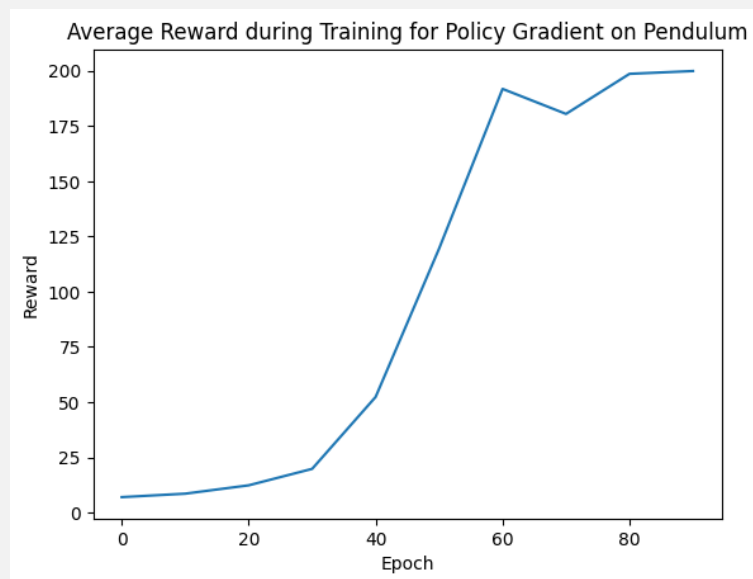
3. Try training policy gradient without subtracting the baseline when computing returns and/or without normalizing the returns. Plot the reward during the training. Report success rate and average reward using the `evaluate` function that is provided to you. How does performance differ?

Without subtracting baseline:

(Final) Success Rate: 0.39

Average Reward (success only): 200.0

Average Reward (all): 181.96

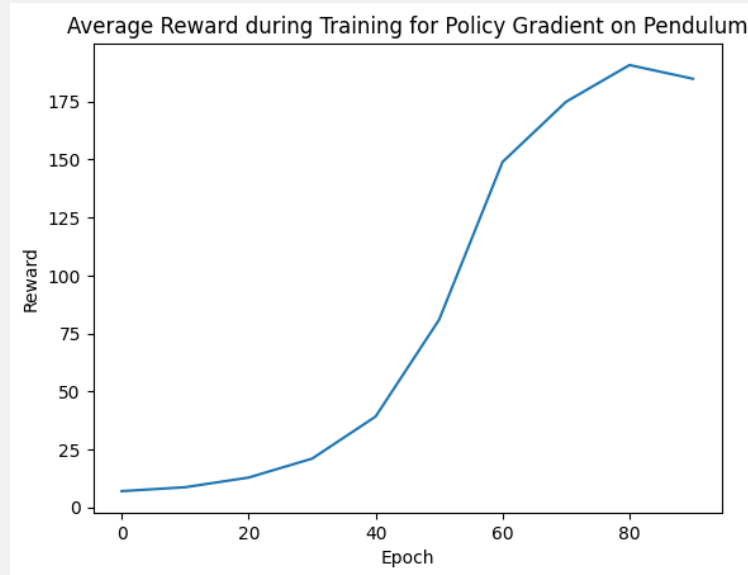
Without normalizing returns:

(Final) Success Rate: 0.60

Average Reward (success only): 200.0

Average Reward (all): 184.05

Without subtracting baseline and without normalizing returns:



(Final) Success Rate: 0.01
Average Reward (success only): 200.0
Average Reward (all): 158.12

As expected, reward and success rate declines in both cases. It seems that, at least for this case, subtracting a baseline is more important than normalizing returns. We didn't talk much about normalizing rewards but we talked extensively about the need to subtract a baseline as a way to help put the results in perspective. Eliminating each of these separately causes a dip in performance, but when we remove both simultaneously, success rate drops to near 0. However, the graphs match more similarly (apart from the fact that the ablation graphs are noisier), which suggests that the model might be overfitting on the training data but failing to generalize to new data.

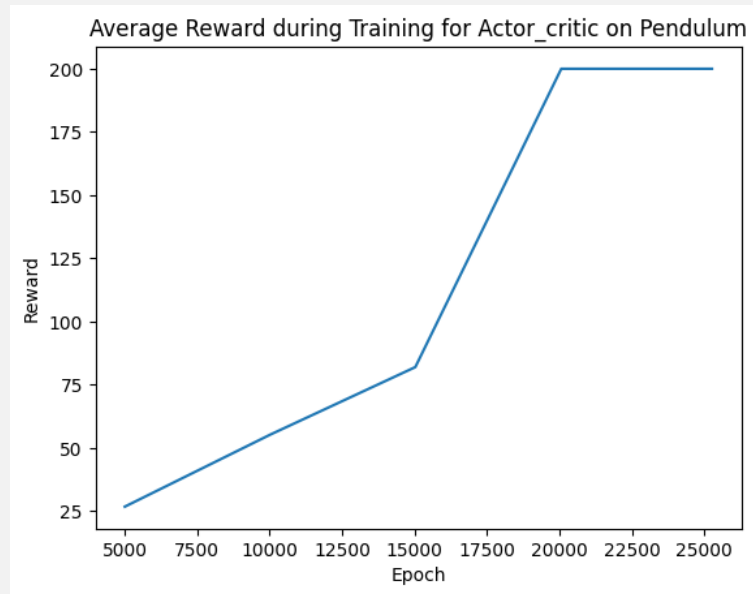
3 Actor Critic

1. Fill in the blanks in the code marked with TODO in `actor_critic.py`.

See Gradescope submission for code.

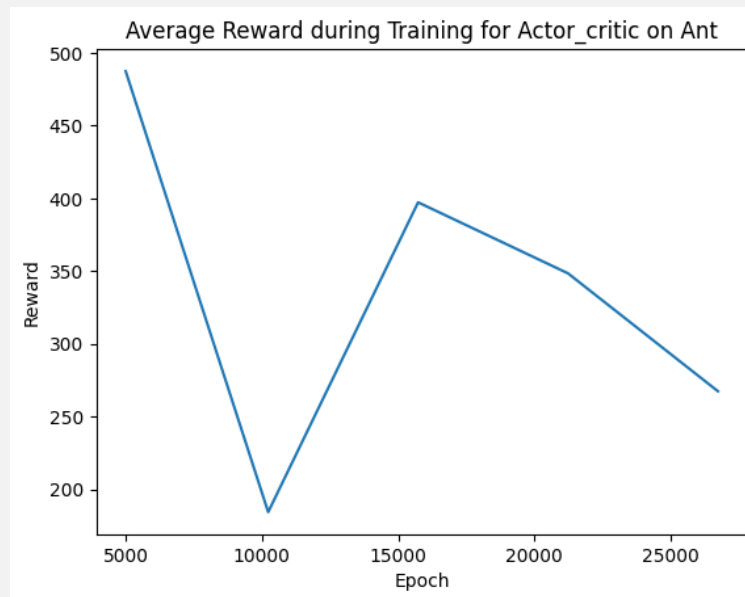
2. Plot the return during the training with default hyperparameters. Report the final average reward using the evaluate function that is provided to you.

Environment: Pendulum



(Final) Average Reward: 189.68

Environment: Ant

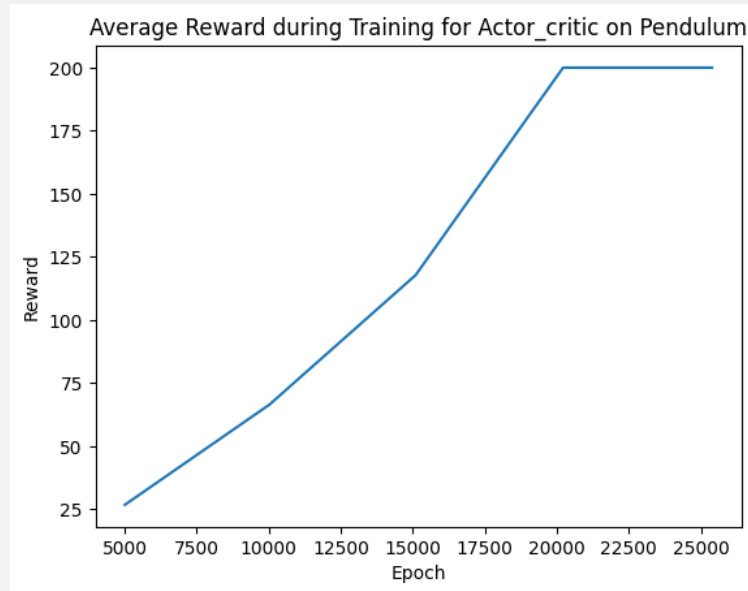


(Final) Average Reward: 330.41

As discussed in Office Hours, the ant environment is more challenging, and can often be glitchy or unstable for simple actor-critic. We will increase the number of train steps for AC to 100,000 in the third experiment for Part 4.

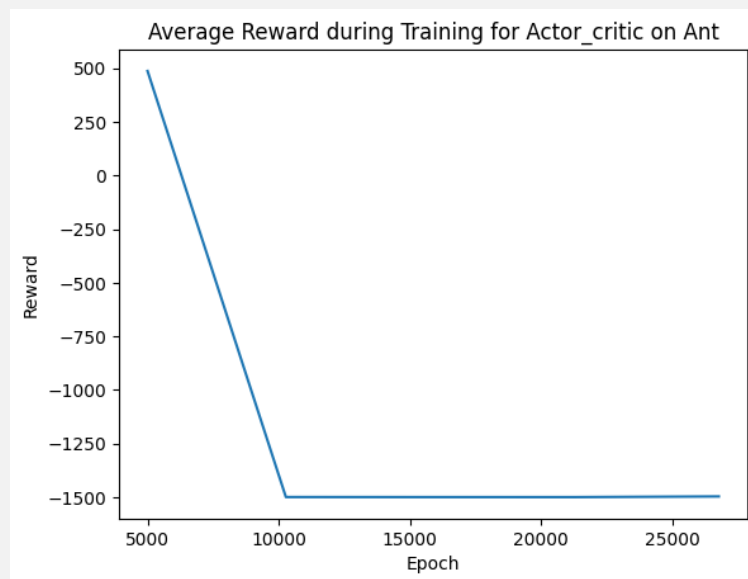
3. Try training actor critic where target Q-values are calculated not through a target Q-network but the

current Q-network. Plot the reward during the training. Report success rate and average reward using the evaluate function that is provided to you. How does performance differ?



Success Rate: 0.00

(Final) Average Reward: 97.27



Success Rate: 0.00

(Final) Average Reward: -1499.50

In both cases, the success rate and final rewards dropped to zero when using the current Q-network as the target Q-network. We discussed in class that it's best not to directly update the target each time the current network changes. Doing so often leads to unstable behavior known as the "moving target problem". Instead, alternative approaches such as moving the target gently via Polyak averaging, or fixing it for certain iterations work better in practice.

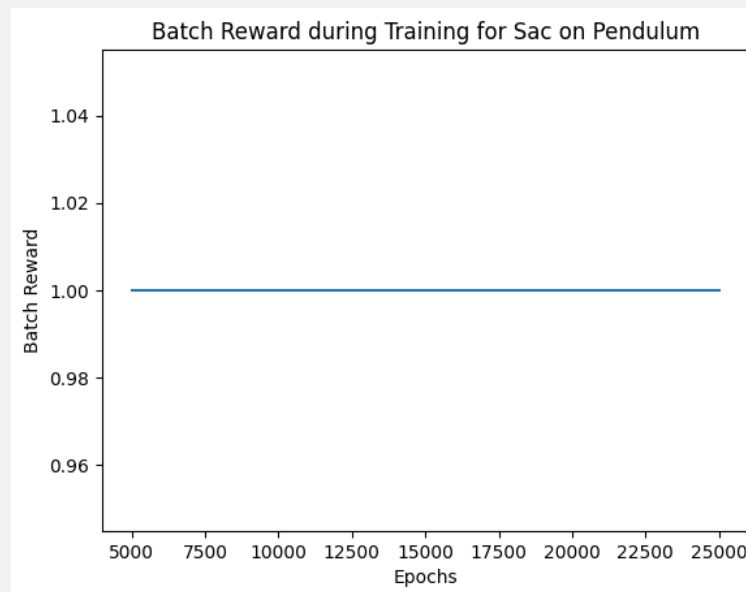
4 Soft Actor Critic

1. Fill in the blanks in the code marked with TODO in `sac.py`. We have provided Psuedocode in the appendix similar to the other problems.

See Gradescope submission for code.

2. Plot the batch reward during training on the pendulum env with the default hyperparameters. Report the average reward on your final trained policy using the evaluate function provided to you.

Environment: Pendulum

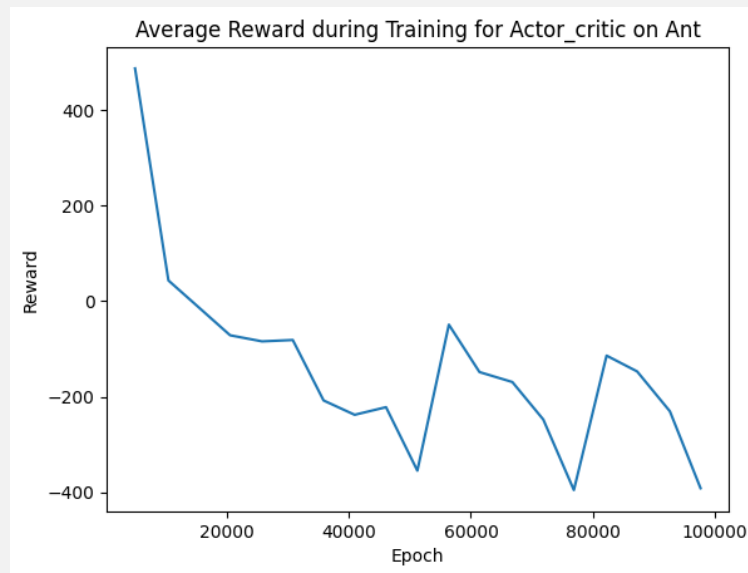


(Final) Average Reward: 200.0

As discussed in OH, a batch reward of 1.0 constantly is consistent with our expected behavior. The pendulum environment is relatively easy for a soft actor critic, so we expect the reward to peak early in training and remain there for the duration.

3. We have also included the Ant environment, a much harder control problem than the pendulum. Increase the number of train steps for AC and for SAC to 100,000. Run both of these algorithms on the ant environment and compare the results of the two methods. Explain why you think the performance differs and why this is the case.

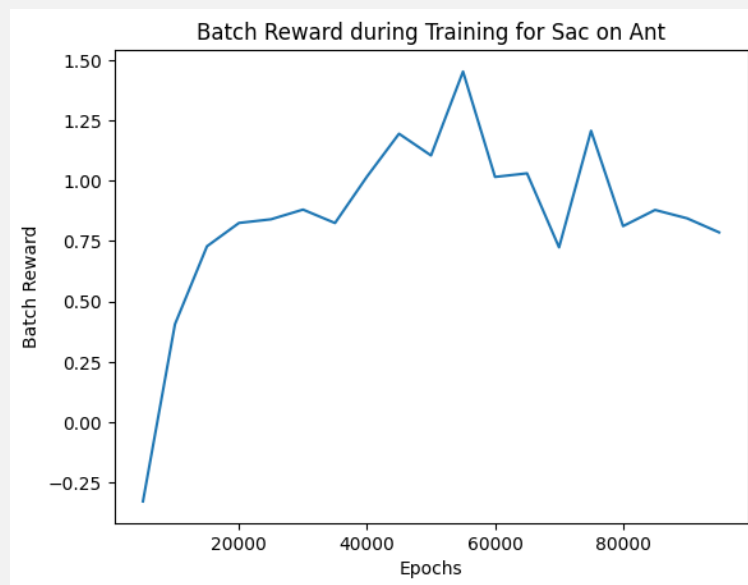
Actor Critic



Average Reward: -274.81

Success Rate: 0.00

Soft Actor Critic



Average Reward: 625.29

Success Rate: 0.99

For the Actor-Critic policy on Ant, the reward and success rate dropped significantly when training to 100,000 steps compared to 30,000. Conversely, the success rate stayed more or less consistent (1.0 vs. 0.99) and the average reward jumped (459.37 to 625.29) for the Soft Actor Critic.

For the standard actor-critic policy, the performance was already starting to drop on more iterations, and the trend simply continuous. As discussed in Office Hours, the ant environment is unstable for standard actor critic. I imagine that some combination of hyperparameter sensitivity, policy degradation, moving target, etc. is causing this noisy downgrade in behavior over time.

However, the soft actor-critic policy did not see any significant drop. Since the goal of soft actor critic is to add an entropy term to encourage exploration, and the ant environment is complex and high-dimension, I believe that the policy was still learning for both 30,000 and 100,000 iterations. As the bar to pass was low, both of these iterations were enough to succeed, but the model still has plenty to learn, which is why the performance hasn't fallen.

5 Discussion

1. Describe the role of the value function in actor-critic methods and how it differs from policy gradient methods.

In both actor-critic and policy gradient methods, the value function is used to reduce variance and stabilize training. For policy gradient methods, the value function is used as a baseline to help understand the actions *relative* to others. It is also used in generalized advantage estimation in the geometric sum, which reduces variance at the cost of additional bias. The difference between the rewards from the current action and the baseline is called advantage, which helps compare and re-weight the distribution of actions in the policy.

In actor-critic methods, the value function is used to compute the Q-function (aka *critic*), which evaluates the actions taken by the policy in real-time. It is learned, and adjusts based on the current state and actions. By considering an expectation over all possible actions/states, it also helps to reduce variance, and guide the policy towards optimal actions.

In standard policy gradient, the value function is not strictly necessary, but rather used as a baseline to help reduce variance. However, in actor-critic the value function is constantly updated, and directly changes the gradient of the policy over time. Standard-policy gradient uses the cumulative rewards in the future with the value function, whereas actor-critic uses the value function to predict future rewards, and adjust the policy.

2. Discuss the advantages and disadvantages of policy gradient, actor-critic, and soft-actor-critic methods in terms of sample efficiency, stability, and convergence properties.

Standard policy gradient is not sample efficient, stable, or easy to converge. As an online policy, it cannot use prior data to adjust the policy, and is therefore not efficient with its samples. Additionally, policy gradient suffers from high variance as it couples together all actions along a single sample, and is therefore unstable, requiring large sample and batch sizes. There are techniques used to mitigate these issues (e.g. natural gradient, importance sampling), but nonetheless they still are often unstable. We even discussed a paper which argued that the little things such as hyperparameters, sampling technique, etc. matter a lot at ensuring convergence.

Actor-critic methods are off-policy, and are therefore able to reuse past actions. Using a replay buffer to store past data and sample independently from there, AC is much more sample efficient compare to PG. Additionally, the Q-function is an expectation of reward over many similar

initial states, and therefore helps to reduce variance. However, standard AC methods oftentimes have issues with convergence. They might converge prematurely, or not at all due to lack of exploration.

Soft-actor critic methods add an entropy term to encourage exploration. As such, these methods maintain the same benefits as standard AC methods (sample efficient, low variance), but also often converge better in practice by smoothening the optimization landscape.